

开发规范

- filename: development-specifications.md
 - title: Campus Activity Platform 开发规范
 - status: active
 - last_updated_at: 2026-04-26 by 陈奕诺
 - description: 校园活动申请、报名与签到平台开发规范，包含技术栈、项目结构、Git 规范、代码规范、API 设计、测试、安全与文档管理等内容。
 - repository: https://github.com/MgSO4-7H2O/Campus_Activity_Platform
-

Campus Activity Platform 开发规范

本文档参考所给《开发规范.pdf》的结构，保留其通用工程规范条目，并根据“校园活动管理与报名签到平台”的需求，将业务模块、数据对象、权限范围和流程规范替换为本项目内容。原开发规范覆盖技术栈、项目结构、Git、API、代码、测试、安全与文档管理等条目；本项目需求明确包含活动申请、分级审核、活动招募、报名、签到、通知、新闻与结项管理等核心功能。

一、项目概述

项目信息	详情
项目名称	Campus Activity Platform（校园活动管理与报名签到平台）
项目仓库	https://github.com/MgSO4-7H2O/Campus_Activity_Platform
项目定位	面向高校校园活动管理场景，实现活动申请、材料提交、分级审核、活动招募、学生报名、签到管理、通知发布、新闻展示、结项管理等功能
主要用户	学生用户、活动负责人、一级审核管理员、二级及以上审核管理员、系统管理员
架构模式	前后端分离，Monorepo 管理
包管理	pnpm 10
核心目标	将线下或半人工的活动审批、报名、签到、结项流程平台化、规范化、可追踪化

1.1 核心业务主线

1. 活动申请与审批：活动负责人提交活动申请和材料，系统根据组织号匹配审核链并逐级流转。
 2. 活动招募与报名：管理员发布招募活动，学生报名并提交材料，管理员审核后形成参与名单。
 3. 活动执行与签到：活动负责人或管理员生成二维码/签到码，学生签到，系统记录并统计。
 4. 结项与归档：活动结束后提交结项材料，管理员审核后归档。
 5. 信息发布与消息提醒：发布新闻、通知、系统公告和流程提醒。
-

二、技术栈

2.1 前端技术

技术	版本	说明
React	19	前端框架
TypeScript	5.7	类型系统
Vite	6	构建工具
Ant Design	5	UI 组件库
Zustand	5	客户端状态管理
TanStack Query	5	服务端状态管理
React Router	7	路由管理
Axios	1.7	HTTP 客户端
Zod	3	Schema 校验
Day.js	1.11	日期时间处理

2.2 后端技术

技术	版本	说明
Node.js	22	运行时
Express	5	Web 框架
TypeScript	5.7	类型系统
Prisma	6	ORM
PostgreSQL	18	主数据库
Redis	7	缓存、会话、限流、签到码临时状态
JWT	9	认证
bcryptjs	3	密码加密
Zod	3	请求参数校验
Nodemailer	8	邮件通知，预留短信/微信通知扩展

2.3 开发工具

工具	用途
ESLint	代码检查
Prettier	代码格式化
Husky	Git hooks
lint-staged	暂存区检查
commitlint	提交信息规范
Vitest	单元测试
Playwright	E2E 测试
Docker Compose	本地容器化开发环境
Makefile	常用命令封装
Prisma Studio / Adminer	数据库调试

三、项目结构

3.1 总体目录结构

```
代码块

1  Campus_Activity_Platform/
2  |— frontend/                                # @cap/web - 前端应用
3  |   |— src/
4  |   |   |— modules/                        # 业务模块
5  |   |   |   |— auth/                      # 登录、认证、个人信息
6  |   |   |   |— portal/                    # 首页、新闻、通知、活动大厅
7  |   |   |   |— activity-application/      # 活动申请与材料提交
8  |   |   |   |— approval-workflow/        # 审核待办、审核流转、流程追踪
9  |   |   |   |— recruitment/              # 活动招募发布
10 |   |   |   |— signup/                   # 学生报名与报名审核
11 |   |   |   |— checkin/                  # 签到、签退、签到大屏
12 |   |   |   |— closure/                  # 结项申请与归档
13 |   |   |   |— notification/             # 站内消息、公开通知、流程提醒
14 |   |   |   |— system-admin/             # 用户、组织号、审核链、系统参数
15 |   |   |— shared/                       # 公共代码
16 |   |   |   |— components/                # 通用 UI 组件
17 |   |   |   |— hooks/                    # 通用 hooks
18 |   |   |   |— utils/                    # 工具函数
19 |   |   |   |— stores/                   # Zustand stores
20 |   |   |   |— constants/                # 常量定义
21 |   |   |   |— layouts/                  # 页面布局
22 |   |   |   |— types/                    # 前端公共类型
```

```

23 | | | └─ __tests__/ # 前端单元测试
24 | | └─ tests/ # Playwright E2E 测试
25 | | └─ public/ # 静态资源
26 | | └─ package.json
27 |
28 | └─ backend/ # @cap/server - 后端服务
29 | | └─ src/
30 | | | └─ modules/ # 业务模块
31 | | | | └─ auth/ # 认证、Token、权限
32 | | | | └─ users/ # 用户与角色
33 | | | | └─ organizations/ # 组织与组织号
34 | | | | └─ approval-chains/ # 审核链配置
35 | | | | └─ applications/ # 活动申请
36 | | | | └─ approvals/ # 审核记录与待办
37 | | | | └─ recruitments/ # 招募活动
38 | | | | └─ signups/ # 报名记录
39 | | | | └─ checkins/ # 签到记录与签到码
40 | | | | └─ closures/ # 结项申请
41 | | | | └─ announcements/ # 新闻通知
42 | | | | └─ notifications/ # 消息推送
43 | | | | └─ system-logs/ # 系统日志
44 | | | └─ shared/
45 | | | | └─ middleware/ # 中间件
46 | | | | └─ utils/ # 工具函数
47 | | | | └─ errors/ # 统一异常
48 | | | | └─ prisma/ # Prisma 客户端
49 | | | | └─ validators/ # 通用校验
50 | | | └─ config/ # 配置文件
51 | | | └─ types/ # 类型声明
52 | | | └─ __tests__/ # 后端单元测试
53 | | └─ prisma/
54 | | | └─ schema.prisma # 数据库 Schema
55 | | | └─ seed.ts # 种子数据
56 | | | └─ migrations/ # 数据库迁移
57 | | └─ dist/ # 编译输出
58 | | └─ package.json
59 |
60 | └─ shared/ # @cap/shared - 前后端共享类型
61 | | └─ src/
62 | | | └─ dto/ # API DTO 类型
63 | | | └─ enums/ # 共享枚举
64 | | | └─ constants/ # 共享常量
65 | | └─ package.json
66 |
67 | └─ docs/
68 | | └─ project-requirements.md # 项目需求
69 | | └─ development-specifications.md # 开发规范

```

```

70 |   |─ database-design.md          # 数据库设计
71 |   |─ deployment.md             # 部署说明
72 |   └─ apis/
73 |       |─ shared.md             # 公共接口
74 |       |─ auth.md
75 |       |─ applications.md
76 |       |─ approvals.md
77 |       |─ recruitments.md
78 |       |─ signups.md
79 |       |─ checkins.md
80 |       |─ closures.md
81 |       |─ announcements.md
82 |       └─ system-admin.md
83 |
84 |─ docker-compose.yml
85 |─ Makefile
86 |─ pnpm-workspace.yaml
87 |─ package.json
88 |─ tsconfig.json
89 |─ README.md

```

3.2 前端模块结构规范

代码块

```

1  modules/activity-application/
2  |─ components/          # 模块专用组件
3  |─ pages/               # 页面组件
4  |─ api/                 # API 请求封装
5  |─ hooks/               # 模块专用 hooks
6  |─ schemas/             # 表单校验 schema
7  |─ types/               # 模块类型定义
8  |─ constants/           # 状态、选项、枚举映射
9  └─ index.ts             # 模块导出

```

3.3 后端模块结构规范

代码块

```

1  modules/applications/
2  |─ application.controller.ts  # 控制器：处理 HTTP 请求
3  |─ application.service.ts    # 服务：业务逻辑
4  |─ application.routes.ts     # 路由定义
5  |─ application.repository.ts  # 数据访问，可选
6  |─ application.schemas.ts   # Zod 请求校验

```

7

|— application.types.ts

模块类型定义

8

|— application.constants.ts

模块常量

3.4 模块边界规范

模块	职责	不应承担的职责
auth	登录、登出、Token、权限中间件	不直接处理业务审批
applications	活动申请草稿、提交、材料管理、申请状态	不决定审核链配置
approval-chains	组织号与审核链映射	不保存具体申请审核意见
approvals	待办、审核意见、流转记录	不处理报名审核以外的报名表单逻辑
recruitments	招募发布、招募规则、招募状态	不保存学生报名材料详情
signups	报名、报名材料、报名审核	不处理现场签到
checkins	签到码、二维码、手动补签、统计导出	不负责活动申请审批
closures	结项材料、结项审核、归档	不负责活动前置申请
announcements	新闻、通知、公告	不负责点对点流程消息
notifications	站内消息、流程提醒、已读未读	不维护公告正文
system-admin	用户、角色、组织、系统参数、异常流程	不实现普通业务页面逻辑

四、Git 规范

4.1 分支结构

代码块

1

main

生产环境，受保护

2

|

3	—	develop	# 开发主分支，受保护
4			
5		— dev/auth	# 认证与用户模块集成分支
6		— dev/application	# 活动申请与审批模块集成分支
7		— dev/recruitment	# 招募与报名模块集成分支
8		— dev/checkin	# 签到与结项模块集成分支
9		— dev/notification	# 新闻通知与消息模块集成分支
10		— dev/system-admin	# 系统管理模块集成分支
11			
12	—	feat/<scope>-<desc>	# 功能分支
13	—	fix/<scope>-<desc>	# 修复分支
14	—	refactor/<scope>-<desc>	# 重构分支
15	—	docs/<desc>	# 文档分支
16	—	hotfix/<desc>	# 紧急修复，从 main 创建

4.2 分支保护

分支	规则
main	禁止直接推送，只允许从 develop 或 hotfix/* 合并
develop	禁止直接推送，由负责人完成合并
dev/*	模块集成分支，模块负责人负责合并
feat/*	功能开发分支，开发完成后合并到对应 dev/*
hotfix/*	紧急修复分支，修复后同时合并到 main 和 develop

4.3 分支命名

类型	格式	示例
功能	feat/<scope>-<功能>-<日期>	feat/application-create-form-0426
修复	fix/<scope>-<描述>-<日期>	fix/checkin-duplicate-submit-0426
重构	refactor/<scope>-<描述>-<日期>	refactor/approval-flow-service-0426
文档	docs/<描述>-<日期>	docs/api-spec-0426
测试	test/<scope>-<描述>-<日期>	test/signup-review-0426

4.4 Commit 规范

格式：

代码块

```
1 <type>(<scope>): <subject>
```

示例：

代码块

```
1 feat(application): add activity application draft API
2 fix(checkin): prevent duplicated QR code checkin
3 docs(api): update approval workflow API
4 test(signup): add signup review service tests
```

Type 类型

Type	说明	示例
feat	新功能	feat(recruitment): add recruitment publish page
fix	Bug 修复	fix(auth): resolve token refresh failure
refactor	重构，不新增功能	refactor(approval): split workflow service
docs	文档更新	docs: update database design
test	测试相关	test(checkin): add duplicate checkin tests
chore	构建、依赖、工具	chore(deps): update vite
style	格式调整	style(frontend): format components
perf	性能优化	perf(portal): optimize announcement list query

Scope 范围

Scope	说明
auth	登录、认证、权限
user	用户与角色
organization	组织与组织号
approval-chain	审核链配置
application	活动申请
approval	审核待办与流程流转
recruitment	活动招募
signup	学生报名
checkin	活动签到
closure	结项归档
announcement	新闻通知
notification	站内消息
system	系统管理
shared	共享代码
db	数据库相关
deps	依赖更新
docs	文档

4.5 工作流程

代码块

```
1  # 1. 从对应 dev 分支创建功能分支
2  git checkout dev/application
3  git pull origin dev/application
4  git checkout -b feat/application-create-form-0426
5
6  # 2. 开发并提交
7  git add .
8  git commit -m "feat(application): add activity application form"
9  git push origin feat/application-create-form-0426
10
11 # 3. 合并到对应 dev 分支
12 git checkout dev/application
13 git pull origin dev/application
14 git merge feat/application-create-form-0426 --no-ff
15 git push origin dev/application
16
17 # 4. 模块负责人合并 dev/* 到 develop
```

```
18 git checkout develop
19 git pull origin develop
20 git merge dev/application --no-ff
21 git push origin develop
22
23 # 5. 发布时从 develop 合并到 main
24 git checkout main
25 git pull origin main
26 git merge develop --no-ff
27 git push origin main
```

4.6 合并前检查清单

合并前必须确认：

1. `pnpm lint` 通过。
2. `pnpm test` 通过。
3. 涉及数据库结构变更时，已提交 Prisma migration。
4. 涉及 API 变更时，已更新 `docs/apis/`。
5. 涉及业务流程变更时，已更新需求或设计文档。
6. 涉及敏感操作时，已补充系统日志记录。
7. 涉及页面交互时，已完成基本可用性自测。

五、API 规范

5.1 RESTful 路径规范

所有接口统一使用：

代码块

```
1 /api/v1/<resource>
```

示例：

方法	路径	说明
GET	/api/v1/ applications	获取活动申请列表
GET	/api/v1/ applications/:id	获取活动申请详情
POST	/api/v1/ applications	创建活动申请草稿
PATCH	/api/v1/ applications/:id	更新活动申请
POST	/api/v1/ applications/:id /submit	提交活动申请
POST	/api/v1/ approvals/:tod old/review	提交审核结果
POST	/api/v1/ recruitments	发布招募
POST	/api/v1/ recruitments/:i d/signup	学生报名
POST	/api/v1/ checkins/ sessions	创建签到场次
POST	/api/v1/ checkins/ submit	学生签到
POST	/api/v1/ closures	提交结项申请
POST	/api/v1/ announcement s	发布新闻通知

5.2 资源命名规范

资源	路径
用户	/users
角色	/roles
组织	/organizations
审核链	/approval-chains
活动申请	/applications
审核待办	/approval-todos
审核记录	/approval-records
招募活动	/recruitments
报名记录	/signups
签到场次	/checkin-sessions
签到记录	/checkin-records
结项申请	/closures
新闻通知	/announcements
站内消息	/notifications
系统日志	/system-logs

5.3 成功响应格式

代码块

```
1  {
2    "code": 200,
3    "message": "success",
4    "data": {}
5  }
```

5.4 分页响应格式

代码块

```
1  {
2    "code": 200,
3    "message": "success",
4    "data": {
```

```
5     "items": [],
6     "pagination": {
7         "page": 1,
8         "page_size": 20,
9         "total": 100,
10        "total_pages": 5
11    }
12 },
13 "request_id": "req_abc123"
14 }
```

5.5 错误响应格式

代码块

```
1  {
2      "code": 40001,
3      "message": "参数校验失败",
4      "errors": [
5          {
6              "field": "activityName",
7              "message": "活动名称不能为空"
8          }
9      ],
10     "request_id": "req_abc123"
11 }
```

5.6 HTTP 状态码

状态码	说明
200	成功
201	创建成功
204	删除成功，无响应体
400	参数错误
401	未认证
403	无权限
404	资源不存在
409	资源冲突
422	业务规则不满足
429	请求过于频繁
500	服务器内部错误

5.7 业务错误码

错误码	说明
40001	参数校验失败
40101	未登录或 Token 失效
40301	当前用户无权限
40401	资源不存在
40901	数据冲突
42201	当前状态不允许该操作
42202	组织号未匹配到审核链
42203	报名已截止
42204	招募名额已满
42205	重复报名
42206	重复签到
42207	签到码无效或已过期
50001	系统内部错误

5.8 核心接口示例

提交活动申请


```
1 POST /api/v1/applications/:id/submit
2 Authorization: Bearer <token>
```

响应:

代码块

```
1  {
2    "code": 200,
3    "message": "success",
4    "data": {
5      "application_id": "app_001",
6      "status": "SUBMITTED",
7      "current_node": "LEVEL_1_REVIEW",
8      "todo_id": "todo_001"
9    }
10 }
```

审核活动申请

代码块

```
1 POST /api/v1/approval-todos/:todoId/review
2 Authorization: Bearer <token>
3 Content-Type: application/json
```

请求:

代码块

```
1  {
2    "action": "APPROVE",
3    "comment": "材料齐全, 活动内容符合要求"
4  }
```

响应:

代码块

```
1  {
2    "code": 200,
3    "message": "success",
4    "data": {
5      "application_id": "app_001",
```

```
6     "status": "LEVEL_2_REVIEWING",
7     "next_todo_id": "todo_002"
8   }
9 }
```

学生报名

代码块

```
1  POST /api/v1/recruitments/:id/signup
2  Authorization: Bearer <token>
3  Content-Type: application/json
```

请求：

代码块

```
1  {
2    "form_values": {
3      "reason": "希望参与校园志愿活动"
4    },
5    "attachment_ids": ["file_001", "file_002"]
6  }
```

学生签到

代码块

```
1  POST /api/v1/checkins/submit
2  Authorization: Bearer <token>
3  Content-Type: application/json
```

请求：

代码块

```
1  {
2    "session_id": "checkin_session_001",
3    "code": "839214",
4    "method": "CODE"
5  }
```

六、代码规范

6.1 命名规范

类型	规范	示例
变量	camelCase	activityName
函数	camelCase	getApplicationById
类	PascalCase	ApplicationService
React 组件	PascalCase	ActivityCard
文件	kebab-case	activity-card.tsx
常量	UPPER_SNAKE_CASE	MAX_UPLOAD_SIZE
类型	PascalCase	ApplicationDetail
接口	PascalCase	ApplicationRepository
枚举	PascalCase	ApplicationStatus
数据库字段	camelCase	createdAt
API JSON 字段	snake_case	application_id

6.2 TypeScript 规范

- 1. 禁止使用隐式 `any`。
- 2. 公共函数必须显式声明参数和返回值类型。
- 3. DTO 类型优先放入 `shared/src/dto`。
- 4. 业务枚举统一放入 `shared/src/enums`。
- 5. 复杂业务状态必须使用枚举，不得使用散落字符串。
- 6. 不允许在业务代码中直接写魔法数字、魔法字符串。

示例：

代码块

```
1  export enum ApplicationStatus {
2    DRAFT = 'DRAFT',
3    SUBMITTED = 'SUBMITTED',
4    LEVEL_1_REVIEWING = 'LEVEL_1_REVIEWING',
5    LEVEL_2_REVIEWING = 'LEVEL_2_REVIEWING',
6    APPROVED = 'APPROVED',
```

```
7     REJECTED = 'REJECTED',
8     NEED_SUPPLEMENT = 'NEED_SUPPLEMENT',
9     ARCHIVED = 'ARCHIVED',
10 }
11
12 export interface ApplicationDetail {
13     id: string
14     activityName: string
15     organizationCode: string
16     status: ApplicationStatus
17     applicantId: string
18     createdAt: string
19 }
```

6.3 后端代码规范

控制器只负责：

1. 获取请求参数。
2. 调用 Zod schema 校验。
3. 调用 service。
4. 返回统一响应。

业务逻辑必须放在 service，不允许在 controller 中写复杂业务判断。

代码块

```
1 export class ApplicationController {
2     async submit(req: Request, res: Response) {
3         const params = submitApplicationParamsSchema.parse(req.params)
4         const user = req.user
5
6         const result = await applicationService.submitApplication({
7             applicationId: params.id,
8             operatorId: user.id,
9         })
10
11         res.success(result)
12     }
13 }
```

Service 负责业务规则：

代码块

```

1  export class ApplicationService {
2      async submitApplication(input: SubmitApplicationInput) {
3          const application = await this.repository.findById(input.applicationId)
4
5          if (!application) {
6              throw new NotFoundError('活动申请不存在')
7          }
8
9          if (application.status !== ApplicationStatus.DRAFT) {
10             throw new BusinessError('当前状态不允许提交')
11         }
12
13         const chain = await approvalChainService.findByOrganizationCode(
14             application.organizationCode,
15         )
16
17         if (!chain) {
18             throw new BusinessError('组织号未匹配到审核链')
19         }
20
21         return this.workflowService.startApproval(application, chain)
22     }
23 }

```

6.4 Prisma Schema 规范

1. 所有表必须包含 `id`、`createdAt`、`updatedAt`。
2. 所有状态字段必须使用 enum。
3. 高频查询字段必须建立索引。
4. 涉及审计的关键业务不得物理删除，使用软删除或状态归档。
5. 审核记录、签到记录、系统日志原则上不可修改，只允许追加。

示例：

代码块

```

1  model ActivityApplication {
2      /// 活动申请唯一标识
3      id String @id @default(uuid())
4
5      /// 申请人
6      applicantId String
7
8      /// 活动名称
9      activityName String

```

```

10
11    /// 活动类型
12    activityType ActivityType
13
14    /// 组织号，用于匹配审核链
15    organizationCode String
16
17    /// 当前申请状态
18    status ApplicationStatus @default(DRAFT)
19
20    /// 当前审核节点
21    currentNodeId String?
22
23    /// 创建时间
24    createdAt DateTime @default(now())
25
26    /// 更新时间
27    updatedAt DateTime @updatedAt
28
29    @@index([applicantId])
30    @@index([organizationCode])
31    @@index([status])
32    @@index([createdAt])
33 }

```

6.5 React 组件规范

1. 页面组件放在 `pages/`。
2. 业务组件放在模块 `components/`。
3. 跨模块复用组件放在 `shared/components/`。
4. Props 必须定义类型。
5. 组件内部不得直接拼接 API URL，应通过模块 `api/` 调用。
6. 表单校验应使用 Zod 或统一表单规则。

代码块

```

1  interface ApplicationStatusTagProps {
2      status: ApplicationStatus
3  }
4
5  export function ApplicationStatusTag({ status }: ApplicationStatusTagProps) {
6      return <Tag color={APPLICATION_STATUS_COLOR_MAP[status]}>
7          {APPLICATION_STATUS_TEXT_MAP[status]}
8      </Tag>

```

```
9 }
```

6.6 前端请求规范

代码块

```
1 export async function getApplicationDetail(id: string) {
2   return request.get<ApplicationDetailResponse>(`/applications/${id}`)
3 }
```

TanStack Query key 规范：

代码块

```
1 export const applicationQueryKeys = {
2   all: ['applications'] as const,
3   list: (params: ApplicationListParams) =>
4     ['applications', 'list', params] as const,
5   detail: (id: string) => ['applications', 'detail', id] as const,
6 }
```

6.7 注释规范

以下内容必须写注释：

1. 审核流转规则。
2. 签到码生成与过期规则。
3. 权限判断逻辑。
4. 状态机转换逻辑。
5. 数据库事务处理逻辑。
6. 非显然的性能优化逻辑。

不推荐注释：

1. 重复解释代码字面含义。
2. 与代码不一致的历史说明。
3. 无实际信息量的模板注释。

七、业务状态规范

7.1 活动申请状态

状态	含义	可执行操作
DRAFT	草稿	编辑、提交、删除
SUBMITTED	已提交	查看、撤回，视业务规则决定
LEVEL_1_REVIEWING	一级审核中	管理员审核
LEVEL_2_REVIEWING	二级审核中	管理员审核
NEED_SUPPLEMENT	待补充材料	申请人补充后重新提交
APPROVED	审核通过	发布招募、发起签到、结项
REJECTED	审核驳回	查看原因、复制后重新申请
ARCHIVED	已归档	只读查看

7.2 报名状态

状态	含义
NOT_SIGNED_UP	未报名
PENDING_REVIEW	已报名待审核
APPROVED	报名通过
REJECTED	报名驳回
NEED_SUPPLEMENT	待补充报名材料
CANCELED	已取消

7.3 签到状态

状态	含义
NOT_CHECKED_IN	未签到
CHECKED_IN	已签到
LATE	迟到
MANUAL_CHECKED_IN	管理员手动补签
ABSENT	缺勤

7.4 结项状态

状态	含义
NOT_STARTED	未发起
DRAFT	草稿
SUBMITTED	已提交
REVIEWING	审核中
NEED_SUPPLEMENT	待补充
APPROVED	结项通过
REJECTED	结项驳回
ARCHIVED	已归档

7.5 状态流转原则

1. 状态只能由服务端计算，前端不得直接决定最终状态。
2. 所有状态流转必须记录操作人、操作时间、操作来源和操作理由。
3. 审核、报名审核、结项审核必须在事务中完成。
4. 非法状态流转返回 42201。
5. 流程异常必须进入异常队列，不得静默失败。

八、权限规范

8.1 角色定义

角色	说明
STUDENT	学生用户
ACTIVITY_OWNER	活动申请负责人
LEVEL_1_ADMIN	一级审核管理员
LEVEL_2_ADMIN	二级及以上审核管理员
SYSTEM_ADMIN	系统管理员

8.2 权限矩阵

功能	学生	活动负责人	一级审核管理员	二级及以上管理员	系统管理员
浏览新闻通知	✓	✓	✓	✓	✓
浏览招募活动	✓	✓	✓	✓	✓
提交报名	✓	✓	✗	✗	✗
活动签到	✓	✓	✗	✗	✗
发起活动申请	✗	✓	✗	✗	✓
查看本人申请进度	✗	✓	✗	✗	✓
审核活动申请	✗	✗	✓	✓	✓
发布招募	✗	✓/受限	✓	✓	✓
审核报名	✗	✓/受限	✓	✓	✓
发起签到	✗	✓	✓	✓	✓
手动补签	✗	✓/受限	✓	✓	✓
发起结项	✗	✓	✗	✗	✓
审核结项	✗	✗	✓	✓	✓
维护组织号	✗	✗	✗	✗	✓
配置审核链	✗	✗	✗	✗	✓
查看系统日志	✗	✗	✗	✗	✓

8.3 权限实现原则

1. 前端只做交互层面的按钮隐藏和路由保护。
2. 后端必须进行最终权限校验。
3. 数据权限必须细化到资源级别，例如申请人只能查看自己的申请，审核管理员只能处理分配给自己的待办。
4. 审核意见、附件材料、系统日志属于敏感数据，必须限制访问。

5. 系统管理员可处理异常流程，但操作必须记录日志。

九、开发环境

9.1 环境说明

项目使用 Docker Compose 提供本地开发环境，包含前端、后端、PostgreSQL、Redis、Adminer 等服务。

9.2 服务地址

服务	地址
前端	http://localhost:5173
后端 API	http://localhost:3000
API Base URL	http://localhost:3000/api/v1
Adminer	http://localhost:8080
PostgreSQL	localhost:5432
Redis	localhost:6379

9.3 测试账号

账号	密码	角色
admin	Admin123	系统管理员
reviewer1	Reviewer123	一级审核管理员
reviewer2	Reviewer123	二级审核管理员
owner	Owner123	活动负责人
student	Student123	学生

9.4 环境变量规范

`.env.example` 必须维护完整，禁止提交真实密钥。

代码块

```
1  NODE_ENV=development
2  PORT=3000
3
```

```
4 DATABASE_URL=postgresql://postgres:postgres@localhost:5432/campus_activity
5 REDIS_URL=redis://localhost:6379
6
7 JWT_ACCESS_SECRET=change-me
8 JWT_REFRESH_SECRET=change-me
9 JWT_ACCESS_EXPIRES_IN=2h
10 JWT_REFRESH_EXPIRES_IN=7d
11
12 UPLOAD_DIR=./uploads
13 MAX_UPLOAD_SIZE_MB=20
14
15 MAIL_HOST=smtp.example.com
16 MAIL_PORT=587
17 MAIL_USER=
18 MAIL_PASS=
19
20 CORS_ORIGIN=http://localhost:5173
```

9.5 常用命令

代码块

```
1 make up # 启动所有服务
2 make down # 停止所有服务
3 make logs # 查看日志
4 make ps # 查看容器状态
5 make shell-server # 进入后端容器
6 make shell-web # 进入前端容器
7
8 pnpm install # 安装依赖
9 pnpm dev # 启动开发环境
10 pnpm build # 构建
11 pnpm lint # Lint 检查
12 pnpm format # 格式化
13 pnpm test # 单元测试
14 pnpm test:e2e # E2E 测试
15
16 pnpm db:migrate # 执行数据库迁移
17 pnpm db:seed # 初始化种子数据
18 pnpm db:studio # 打开 Prisma Studio
```

十、测试规范

10.1 测试策略

测试类型	工具	覆盖范围	覆盖率目标
单元测试	Vitest	工具函数、Service、状态机、权限判断	≥ 80%
组件测试	Vitest + Testing Library	表单、状态标签、业务组件	核心组件覆盖
API 测试	Vitest / Supertest	后端路由与业务规则	核心接口覆盖
E2E 测试	Playwright	登录、申请、审核、报名、签到、结项	关键路径覆盖

10.2 必测关键路径

1. 学生/负责人登录。
2. 活动负责人创建草稿并提交活动申请。
3. 系统根据组织号匹配审核链。
4. 一级管理员审核通过。
5. 二级管理员审核通过。
6. 管理员发布招募。
7. 学生报名并提交材料。
8. 管理员审核报名。
9. 活动负责人创建签到场次。
10. 学生扫码或输入签到码签到。
11. 活动负责人导出签到记录。
12. 活动负责人提交结项申请。
13. 管理员审核结项并归档。

10.3 测试要求

1. 新功能必须同步补充测试。
2. 修复 Bug 时，应先写复现测试。
3. 涉及审核流转、报名名额扣减、签到防重复的逻辑必须有单元测试。
4. 涉及数据库事务的代码必须测试异常回滚场景。
5. 提交前必须保证测试通过。

6. 测试数据不得依赖真实个人信息。

10.4 单元测试示例

代码块

```
1  import { describe, expect, it } from 'vitest'
2  import { ApplicationStatus } from '@cap/shared'
3  import { canSubmitApplication } from '../application-state-machine'
4
5  describe('application state machine', () => {
6    it('should allow draft application to be submitted', () => {
7      expect(canSubmitApplication(ApplicationStatus.DRAFT)).toBe(true)
8    })
9
10   it('should not allow approved application to be submitted again', () => {
11     expect(canSubmitApplication(ApplicationStatus.APPROVED)).toBe(false)
12   })
13 })
```

10.5 测试文件位置

代码块

```
1  # 后端单元测试
2  backend/src/__tests__/
3
4  # 前端单元测试
5  frontend/src/__tests__/
6
7  # 前端 E2E 测试
8  frontend/tests/
9
10 # 模块内测试，也可与源码相邻
11 modules/applications/__tests__/
```

十一、安全规范

11.1 认证与授权

项目	规范
认证方式	JWT Bearer Token
Access Token 过期时间	2h
Refresh Token 过期时间	7d
密码加密	bcrypt, rounds = 10
会话存储	Redis，支持强制登出
权限模型	RBAC + 资源级数据权限
敏感接口	必须校验角色、资源归属和当前流程节点

11.2 安全最佳实践

项目	实践
输入验证	Zod schema 校验所有输入
SQL 注入	Prisma 参数化查询
XSS	前端转义用户输入，富文本白名单过滤
CSRF	JWT 通过 Authorization header 传递，不使用 Cookie 时无需 CSRF
CORS	严格配置白名单
Rate Limit	登录、报名、签到等接口限流
安全头部	Helmet 中间件
文件上传	校验大小、扩展名、MIME 类型，保存随机文件名
附件访问	必须鉴权，禁止直接暴露真实存储路径
日志脱敏	不记录密码、Token、身份证号等敏感信息

11.3 敏感操作日志

以下操作必须记录到 `SystemLog`：

- 1. 登录、登出、Token 刷新失败。
- 2. 密码修改、密码重置。
- 3. 用户创建、删除、禁用、启用。
- 4. 角色分配、角色撤销。
- 5. 组织号新增、修改、删除。
- 6. 审核链配置变更。
- 7. 活动申请提交、撤回、审核。
- 8. 报名审核。
- 9. 手动补签。

10. 结项审核。
11. 系统管理员处理异常流程。
12. 数据导出。
13. 公告发布、修改、删除。

11.4 文件安全规范

1. 上传文件必须绑定业务对象。
2. 文件不得使用用户上传的原始文件名作为存储文件名。
3. 文件下载必须进行权限校验。
4. 申请材料、报名材料、结项材料不得被无关用户访问。
5. 删除业务数据时，附件应进入待清理状态，不得立即不可恢复删除。
6. 文件上传失败必须返回明确错误，不得产生孤立业务记录。

11.5 签到安全规范

1. 签到码必须有过期时间。
 2. 二维码内容不得包含可伪造的敏感业务信息。
 3. 每名学生每个签到场次只能成功签到一次。
 4. 手动补签必须记录补签人、补签原因和补签时间。
 5. 签到接口必须进行限流，防止暴力枚举签到码。
 6. 可预留地理位置校验与设备指纹校验扩展点。
-

十二、性能与可靠性规范

12.1 响应时间要求

项目动作	响应时间	说明
首页进入	< 3s	活动大厅首页加载完成
登录跳转	< 3s	登录后进入对应角色主页
新闻/通知列表查询	< 3s	支持分页
招募活动列表查询	< 3s	支持筛选
活动详情查询	< 3s	包含基础信息与报名状态
审批待办详情展开	< 3s	管理员查看申请详情
管理员审批状态更新	< 3s	负责人端可看到状态变化
学生报名提交	< 3s	名额扣减与报名记录创建
扫码签到响应	< 2s	返回签到成功/失败
签到大屏刷新	< 2s	统计数据刷新
签到记录导出	< 5s	生成 Excel 报表

需求报告中对首页、登录、导航、审批详情、扫码签到、状态更新、签到大屏刷新和报表导出均提出了明确的响应或处理时间要求，可作为本规范的性能基线。

12.2 并发与一致性要求

1. 报名名额扣减必须使用数据库事务或乐观锁。
2. 同一用户对同一招募活动不得重复报名。
3. 同一用户对同一签到场次不得重复签到。
4. 审核待办同一时间只能被有权限的当前处理人处理。
5. 审核通过后生成下一级待办必须与状态更新在同一事务中完成。
6. 组织号无法匹配审核链时，申请进入异常队列，不得丢失。

12.3 数据备份与恢复

1. 开发环境允许手动重置数据库。
2. 测试环境每日备份一次。
3. 生产环境上线前必须制定备份策略。
4. 数据库迁移前必须备份。

5. 影响用户访问的维护操作必须提前通知并保留回滚方案。

十三、文档管理

13.1 文档结构

代码块

```
1 docs/
2   |— project-requirements.md
3   |— development-specifications.md
4   |— database-design.md
5   |— deployment.md
6   |— user-manual.md
7   |— test-report.md
8   |— apis/
9       |— shared.md
10      |— auth.md
11      |— applications.md
12      |— approvals.md
13      |— recruitments.md
14      |— signups.md
15      |— checkins.md
16      |— closures.md
17      |— announcements.md
18      |— system-admin.md
```

13.2 文档分类

类型	内容	维护方式
公共文档	需求、开发规范、数据库设计、部署说明	统一维护，只在develop修改
API 文档	各模块接口说明	模块负责人维护
设计文档	流程设计、状态机设计、权限设计	相关模块维护
用户文档	用户手册、操作说明	发布前统一整理
测试文档	测试计划、测试报告	测试负责人维护

13.3 文档元数据

每个 Markdown 文档必须包含：

代码块

```
1  ---
2  filename: 文档名称.md
3  title: 文档标题
4  status: draft | active | outdated | archived
5  version: 版本号
6  last_updated_at: YYYY-MM-DD
7  last_updated_by: 更新者姓名
8  description: 文档简述
9  ---
```

13.4 文档更新规则

- 1. 公共文档更新必须说明原因。
- 2. API 变更必须同步更新接口文档。
- 3. 数据库字段变更必须同步更新数据库设计文档。
- 4. 流程状态变更必须同步更新开发规范和测试用例。
- 5. 文档废弃后状态改为 `outdated` 或 `archived`，不得直接删除。

十四、发布与部署规范

14.1 环境划分

环境	分支	用途
本地环境	任意开发分支	开发调试
测试环境	develop	集成测试
预发布环境	release/*	发布前验证
生产环境	main	正式运行

14.2 发布前检查

- 1. 所有测试通过。
- 2. 数据库 migration 已验证。
- 3. `.env.example` 已更新。

- 4. API 文档已更新。
- 5. 用户关键路径已完成 E2E 测试。
- 6. 涉及数据结构变更时，已准备回滚方案。
- 7. 影响用户访问时，已准备维护公告。
- 8. 版本号已更新。
- 9. CHANGELOG 已更新。

14.3 版本号规范

采用语义化版本：

代码块

```
1 MAJOR.MINOR.PATCH
```

示例：

版本	说明
1.0.0	初始可用版本
1.1.0	新增功能模块
1.1.1	Bug 修复
2.0.0	不兼容升级

十五、核心业务开发注意事项

15.1 活动申请与审批

- 1. 申请提交前必须校验必填字段和附件完整性。
- 2. 组织号是审核链匹配的关键字段，不允许为空。
- 3. 提交申请后必须生成唯一申请编号。
- 4. 审核链至少包含一级审核人，可扩展多级。
- 5. 审核意见必须保存。
- 6. 驳回或要求补充材料时，必须填写具体原因。
- 7. 所有审核记录必须可追踪、不可随意修改。

15.2 招募与报名

1. 只有审核通过的活动才能发布招募。
2. 招募必须设置报名时间、招募人数和所需材料。
3. 学生报名时必须校验报名截止时间和剩余名额。
4. 报名通过后进入活动参与名单。
5. 报名被驳回或要求补充材料时，系统应发送通知。

15.3 签到管理

1. 只有报名通过的学生可以签到。
2. 签到方式包括二维码、签到码、管理员手动补签。
3. 签到码应定时刷新或设置过期时间。
4. 系统必须防止重复签到。
5. 管理员手动补签必须记录原因。
6. 签到记录应支持导出。

15.4 结项管理

1. 只有已通过审批且已执行的活动才能发起结项。
2. 结项材料包括活动总结、参与名单、活动照片、经费说明、成果材料等。
3. 结项审核流程可复用活动申请审核链，也可使用独立配置。
4. 结项通过后活动进入归档状态。
5. 归档后原则上只读。

15.5 新闻通知与消息推送

1. 公告、新闻、通知应区分类型。
 2. 重要通知支持置顶。
 3. 流程消息应自动触发，如审核通过、审核驳回、材料待补充、报名结果、签到提醒。
 4. 定向推送必须记录接收范围。
 5. 消息应记录已读/未读状态。
-

十六、代码审查清单

16.1 通用检查

- 命名是否符合规范。

- 是否存在重复代码。
- 是否存在隐式 `any`。
- 是否存在魔法字符串或魔法数字。
- 是否有必要的错误处理。
- 是否有必要的日志记录。
- 是否更新相关文档。
- 是否补充测试。

16.2 后端检查

- Controller 是否只负责请求处理。
- Service 是否封装业务逻辑。
- 是否使用 Zod 校验输入。
- 是否进行权限校验。
- 是否处理资源级数据权限。
- 是否使用事务保证一致性。
- 是否避免直接返回敏感字段。
- 是否记录关键操作日志。

16.3 前端检查

- 页面是否区分加载、成功、空状态、错误状态。
- 表单是否有明确校验提示。
- 是否避免在组件中硬编码接口地址。
- 是否正确处理权限按钮展示。
- 是否正确处理分页、筛选、刷新。
- 是否避免重复请求。
- 是否有必要的用户确认弹窗。

16.4 数据库检查

- 是否添加必要索引。
- enum 是否覆盖所有业务状态。
- migration 是否可执行。
- 是否考虑软删除或归档。

- 是否避免破坏已有数据。
 - 是否考虑审计字段。
-

十七、附录：推荐模块开发顺序

1. 初始化 Monorepo、前后端基础工程、数据库连接。
2. 完成用户登录、角色权限、基础布局。
3. 完成组织号与审核链配置。
4. 完成活动申请草稿、提交、附件上传。
5. 完成自动审核人匹配与审核待办。
6. 完成管理员审核流转。
7. 完成招募发布。
8. 完成学生报名与报名审核。
9. 完成签到场次、二维码/签到码、签到记录。
10. 完成结项申请与审核。
11. 完成新闻通知与消息推送。
12. 完成统计导出、系统日志、异常流程处理。
13. 完成测试、文档、部署与验收材料。